

Analysis Results

Objective

Kosmos follow-up — complete the T1 soundness audit (self-contained)

Follow-up to your Once T1 audit. You certified `main.rs`` composition but flagged that the helper module bodies were missing, so the parser / canonicalization / signature-scope logic was not line-audited. Attachments aren't supported here, so **the full source is inlined verbatim below.** Same scope: **prove-independent**; audit **composition and claims**; assume the underlying crates (SHA-256, RSA, Poseidon/BN254) are correct — audit how they're used, not the primitives.

The statement being enforced (SPEC §2)

The proof must convince a verifier of exactly: the prover possesses an email `M` with a DKIM-Signature `H` such that (a) `H` is well-formed per RFC 6376; (b) the signing domain `d`` equals public input `claimed_domain``; (c) the RSA signature verifies against the key over the canonicalized signed portion per `H's `c=~/h=``; (d) nullifier `N = Poseidon(sep, claimed_domain, signature)` is fresh. `v2-A: (c)` uses a **witnessed** key, plus an assertion that `leaf = Poseidon(sep, claimed_domain, selector, n, e)`` is a member of the verifier-pinned `registry_root`` (fixed Merkle depth 20). The host is **untrusted** — anything it can lie about must be re-asserted in-guest.

Guest steps to verify (SPEC §4)

1. Locate `DKIM-Signature`` at `dkim_header_index``; exact case-insensitive `DKIM-Signature:`` match; parse tags.
2. Assert `v=1``, `a=rsa-sha256``, `c=relaxed/relaxed``, `d == claimed_domain``, `s == witnessed selector``, `b~/bh == witness``, and `l=`` (body-length) NOT present.
3. Apply relaxed/relaxed canonicalization.
4. SHA-256 of canonicalized body, assert ``== bh``.
5. Build canonicalized header set per `h=``, DKIM-Signature appended last with its own `b=`` emptied (RFC 6376 §3.7).
6. RSA-PKCS1v1.5-SHA256 verify of `b`` over that set, against the witnessed key.
7. nullifier = `Poseidon(sep, claimed_domain, signature)`; commit it + `claimed_domain + registry_root``.
v1: optional opt-in disclosure of one signed identity header's address.

What to audit, per module

- **dkim.rs`** (§4.1/§4.2) — highest priority. All step-2 assertions present and correct; `l=`` actually rejected. **Adversarial:** with  $\geq 2$  `DKIM-Signature`` headers, can a prover point `dkim_header_index`` at an attacker-favorable one to prove something unintended? Is only the witnessed header considered, and is that safe?
- **canonical.rs`** (§4.3). relaxed/relaxed correctness (header unfold + WSP collapse + case; body CRLF + trailing-empty-line). **Adversarial:** any slack letting a **different** byte string verify under the same signature?

- `body.rs` (§4.4).` Body SHA-256 recomputed in-guest and asserted ``== bh`` (not trusting the witnessed ``bh``).
- `signed_set.rs` (§4.5)` — soundness-critical. Header set per ``h=``; DKIM-Signature last with `only its own `b=`` emptied. Header-prepend defense: is the `*signed*` (covered) instance read, so a prepended unsigned header can't inject? Does ``h_contains`` gate v1 disclosure to headers actually under ``h=``?
- `verify.rs` (§4.6).` RSA verify against the `witnessed` key; no accept on malformed padding / wrong digest.
- `nullifier.rs` (§4.7).` Poseidon(sep, claimed_domain, signature) — per-signature, deterministic, domain-separated from the registry leaf.
- `address.rs` (v1 §4.10).` Address parsed only from the `*signed*` header value; robust to malformed input; cannot be sourced from outside the signed set.
- `registry.rs` / `main.rs`` included again for composition context (already audited).

Specific questions

1. Any canonicalization or multi-header ambiguity that breaks the binding between the verified signature and ``claimed_domain`` / the committed outputs?
2. Is the ``b=`` emptying scoped to only the DKIM-Signature's own ``b=``?
3. Is ``l=`` rejection actually enforced (no body-length-truncation surface)?
4. v1: can a disclosed address ever originate from an unsigned/prepended header?

Output

Per module: sound / gap. For any gap, a concrete reproducible scenario. If clean, an explicit statement that §4.1–§4.7 (+v1 §4.10) enforce the §2 statement with no exploitable host/prover deviation — the complete sign-off the first pass could not give.

Source (verbatim)

methods/guest/src/main.rs

```rust`

`/// Guest program — SPEC.md §4.`

`///`

`/// Each numbered TODO below maps to a step in `SPEC.md` §4 and is filled in`

`/// *in order*. Per guardrails:`

`/// - Implement only what SPEC.md describes. Ask before adding anything else.`

`/// - Stay under 500 lines, hand-readable (SPEC.md §9).`

`/// - The cryptographic statement in SPEC.md §2 is the contract.`

`///`

`/// Pure-function modules carry their own `#[cfg(test)]` unit tests:`

`/// * `canonical`: SPEC.md §4.3 relaxed/relaxed canonicalization`

`/// * `dkim`: SPEC.md §4.1 + §4.2 header location, parse, assertions`

```

//! * `body`: SPEC.md §4.4 body SHA-256 verification
//! * `signed_set`: SPEC.md §4.5 canonicalized signed-header set
//! * `verify`: SPEC.md §4.6 RSA-PKCS1v1.5-SHA256 verify
//! * `nullifier`: SPEC.md §4.7 Poseidon-based replay nullifier
//!
//! Build with the RISC0 toolchain to produce a guest ELF; build natively
//! (`cargo test`) to run those unit tests on the host architecture.

#![cfg_attr(not(test), no_main)]

mod address;
mod body;
mod bytes_util;
mod canonical;
mod dkim;
mod nullifier;
mod signed_set;
mod verify;

#[cfg(not(test))]
risc0_zkvm::guest::entry!(main);

/// Domain separator for the nullifier construction (SPEC.md §3, §4.7).
/// Locked at v0; changing it invalidates every prior proof.
/// Encoded as a single BN254 field element with right-padding when fed into
/// Poseidon. See SPEC.md §4.7.
pub const DOMAIN_SEPARATOR_V0: &[u8] = b"0nce-v0-nullifier";

#[cfg(not(test))]
fn main() {
 use nce_core::{PublicInputs, PublicOutputs, Witness};
 use risc0_zkvm::guest::env;

 // Read inputs. The host writes witness then public inputs in this order;
 // the order is the contract between host and guest.
 let witness: Witness = env::read();
 let public_inputs: PublicInputs = env::read();

 // §4.1 + §4.2: locate, parse, validate the DKIM-Signature header. Any
 // assertion failure (wrong magic, wrong v/a/c, d != claimed_domain,
 // s != selector, b or bh != witness, l= present) panics here.
 let parsed = dkim::locate_and_parse(
 &witness.email_raw,
 witness.dkim_header_index,
 &public_inputs.claimed_domain,
 &witness.selector,
);

```

```

 &witness.signature,
 &witness.body_hash,
);

// §4.4: SHA-256 of canonicalized body, assert == parsed.body_hash.
// (Body canonicalization, §4.3 body half, runs inside verify_body_hash.)
body::verify_body_hash(&witness.email_raw, &parsed.body_hash);

// §4.5: construct canonicalized header set with b= emptied. Uses
// canonical::canonicalize_header_relaxed internally (§4.3 header half).
let signed_data = signed_set::build_signed_data(
 &witness.email_raw,
 &parsed.signed_headers_raw,
 parsed.header_start,
 parsed.header_end,
);

// §4.6' (v2-A): RSA-PKCS1v1.5-SHA256 verify against the WITNESSED key (not
// a public input). The expensive step.
verify::verify_rsa_signature(
 &signed_data,
 &parsed.signature,
 &witness.pubkey_n,
 &witness.pubkey_e,
);

// §4.6a (v2-A): registry membership. Bind the witnessed key to the
// verifier-pinned `registry_root` so a prover cannot supply their own key.
// leaf = Poseidon(sep, claimed_domain, selector, n, e); prove it is a
// member of the tree at `leaf_index`. Failure = panic = no proof. This is
// the v2-A soundness assertion that closes the v0 pubkey-trust gap.
let leaf = nce_core::registry::registry_leaf(
 &public_inputs.claimed_domain,
 &witness.selector,
 &witness.pubkey_n,
 &witness.pubkey_e,
);
assert!(
 nce_core::registry::verify_membership(
 &leaf,
 &witness.merkle_path,
 witness.leaf_index,
 &public_inputs.registry_root,
),
 "signing key is not a member of the pinned registry root"
);

```

```

// §4.7: nullifier = Poseidon(DOMAIN_SEPARATOR_V0, claimed_domain, signature).
let nullifier = nullifier::compute_nullifier(
 DOMAIN_SEPARATOR_V0,
 &public_inputs.claimed_domain,
 &parsed.signature,
);

// §4.9 + §4.10 (v1): identity-header disclosure is OPT-IN. With
// `disclosed_header_kind == None` the guest reveals no address (v0
// privacy-preserving mode): nobody is named unless the prover chose to.
let disclosed_address = match public_inputs.disclosed_header_kind {
 None => Vec::new(),
 Some(kind) => {
 // §4.9: locate the disclosed header WITHIN the signed set. Assert
 // it is covered by `h=` (else the signature does not protect it),
 // then read the bottom-most instance — the one the signature
 // actually covers. Reading the signed instance defeats
 // header-prepend attacks.
 let disclosed_name = kind.header_name();
 assert!(
 signed_set::h_contains(&parsed.signed_headers_raw, disclosed_name),
 "disclosed header is not covered by the DKIM h= tag"
);
 let value = signed_set::signed_header_value(&witness.email_raw, disclosed_name)
 .expect("disclosed header is in h= but absent from the message");

 // §4.10: parse the address from the signed header and emit it.
 // Domain/signer alignment is a verifier-side policy check (see host
 // verify.rs), not a guest assertion — real mail is often signed by
 // a provider whose d= differs from the From domain.
 address::extract_address(&value)
 }
};

// §4.8: commit public outputs (claimed_domain + nullifier + address +
// registry_root echoed so the verifier can re-check the pinned root).
env::commit(&PublicOutputs {
 claimed_domain: public_inputs.claimed_domain,
 nullifier,
 disclosed_address,
 registry_root: public_inputs.registry_root,
});
}
...

```

```
core/src/registry.rs
```

```
``rust
```

```
//! v2-A registry leaf + Merkle membership.
```

```
//!
```

```
//! Shared by the guest (which proves a witnessed DKIM key is a registry
```

```
//! member) and the host (which builds the tree). The leaf is a BN254
```

```
//! Poseidon hash (light-poseidon`, Circom/Iden3 params) of each
```

```
//! variable-length input compressed to one field element via SHA-256 then
```

```
//! reduced mod the field order — same primitive as the nullifier, so it stays
```

```
//! Circom-compatible. The node hash (2-to-1 compression up the tree) is
```

```
//! SHA-256: a BN254 Poseidon node ran as unaccelerated software bignum in the
```

```
//! RISC0 guest and 4×'d prove time (BENCHMARKS.md 2026-06-15); SHA-256 rides
```

```
//! the accelerator and is ~2 orders cheaper. Only the tree shape changed —
```

```
//! leaf semantics are unaffected.
```

```
//!
```

```
//! Leaf semantics are LOCKED at v2-A and must not change in v2-C — C may swap
```

```
//! the tree shape (e.g. to an indexed tree for revocation) but a leaf always
```

```
//! means Poseidon(sep, domain, selector, n, e)`.
```

```
use ark_bn254::Fr;
```

```
use ark_ff::{BigInteger, PrimeField};
```

```
use light_poseidon::{Poseidon, PoseidonHasher};
```

```
use sha2::{Digest, Sha256};
```

```
/// Domain separator for the registry leaf. Locked; distinct from the
```

```
/// nullifier separator so leaves and nullifiers never collide.
```

```
pub const DOMAIN_SEPARATOR_REGISTRY_V2: &[u8] = b"Once-v2-registry-leaf";
```

```
/// Fixed Merkle depth. A path is exactly this many siblings; the guest
```

```
/// rejects any other length (prevents path-length-ambiguity forgeries).
```

```
/// $2^{20} \approx 1.05\text{M}$ keys — ample for any realistic DKIM-key set.
```

```
pub const REGISTRY_DEPTH: usize = 20;
```

```
/// Compress arbitrary-length bytes to one BN254 field element (same scheme as
```

```
/// the nullifier: SHA-256 then reduce mod order).
```

```
fn bytes_to_field(bytes: &[u8]) -> Fr {
```

```
 Fr::from_be_bytes_mod_order(&Sha256::digest(bytes))
```

```
}
```

```
fn fr_to_bytes(f: Fr) -> [u8; 32] {
```

```
 let be = f.into_bigint().to_bytes_be();
```

```
 let mut out = [0u8; 32];
```

```
 out[32 - be.len()..].copy_from_slice(&be);
```

```
 out
```

```

}

/// `leaf` = Poseidon(sep, domain, selector, n, e`.
pub fn registry_leaf(domain: &[u8], selector: &[u8], pubkey_n: &[u8], pubkey_e: &[u8]) -> [u8; 32]
{
 let inputs = [
 bytes_to_field(DOMAIN_SEPARATOR_REGISTRY_V2),
 bytes_to_field(domain),
 bytes_to_field(selector),
 bytes_to_field(pubkey_n),
 bytes_to_field(pubkey_e),
];
 let mut h = Poseidon::<Fr>::new_circom(5).expect("poseidon init t=6");
 fr_to_bytes(h.hash(&inputs).expect("poseidon hash (leaf)"))
}

/// 2-to-1 Merkle compression `node` = SHA-256(left || right)`.
///
/// SHA-256 (not BN254 Poseidon) so the fold rides the RISC0 SHA accelerator
/// instead of unaccelerated bignum — see the module note and BENCHMARKS.md.
/// Untagged 64-byte input matches the original untagged Poseidon(l,r); the
/// fixed tree depth and `leaf_index < 2^DEPTH` bound (see `verify_membership`)
/// defeat leaf/node confusion without a domain byte.
pub fn node_hash(left: &[u8; 32], right: &[u8; 32]) -> [u8; 32] {
 let mut h = Sha256::new();
 h.update(left);
 h.update(right);
 h.finalize().into()
}

/// Domain-separated empty-leaf constant for padding unused slots.
pub fn empty_leaf() -> [u8; 32] {
 let mut h = Poseidon::<Fr>::new_circom(1).expect("poseidon init t=2");
 fr_to_bytes(
 h.hash(&[bytes_to_field(b"0nce-v2-registry-empty")])
 .expect("poseidon hash (empty)"),
)
}

/// Verify that `leaf` at `leaf_index` authenticates to `root` via `path`.
/// `path[i]` is the sibling at level `i` (leaf level = 0). The bit `i` of
/// `leaf_index` selects whether the running node is the left (0) or right (1)
/// child at that level.
pub fn verify_membership(
 leaf: &[u8; 32],
 path: &[[u8; 32]],

```



```

 leaf_index: u32,
 root: &[u8; 32],
) -> bool {
 if path.len() != REGISTRY_DEPTH {
 return false;
 }
 // Witness hygiene: only the low REGISTRY_DEPTH bits of leaf_index are
 // consumed by the fold; reject anything that sets higher bits so the
 // index is unambiguous (no soundness impact — the fold is already pinned
 // by leaf + siblings — but it removes a malleable witness.)
 if (leaf_index as u64) >= (1u64 << REGISTRY_DEPTH) {
 return false;
 }
 let mut node = *leaf;
 for (level, sibling) in path.iter().enumerate() {
 node = if (leaf_index >> level) & 1 == 0 {
 node_hash(&node, sibling)
 } else {
 node_hash(sibling, &node)
 };
 }
 &node == root
}

/// A built fixed-depth Merkle tree over a contiguous prefix of real leaves
/// (indices `0..leaves.len()`), with all remaining slots = `empty_leaf()`.
pub struct RegistryTree {
 /// `layers[0]` = the real-leaf prefix; `layers[REGISTRY_DEPTH]` = `[root]`.
 layers: Vec<Vec<[u8; 32]>>,
 /// Precomputed empty-subtree hash per level (`empties[0] == empty_leaf()`).
 empties: Vec<[u8; 32]>,
}

impl RegistryTree {
 /// Build the tree from real leaves placed at indices `0..leaves.len()`.
 pub fn build(leaves: Vec<[u8; 32]>) -> Self {
 let mut empties = Vec::with_capacity(REGISTRY_DEPTH + 1);
 empties.push(empty_leaf());
 for level in 0..REGISTRY_DEPTH {
 let e = empties[level];
 empties.push(node_hash(&e, &e));
 }

 let mut layers: Vec<Vec<[u8; 32]>> = Vec::with_capacity(REGISTRY_DEPTH + 1);
 layers.push(leaves);
 for level in 0..REGISTRY_DEPTH {

```

```

 let cur = &layers[level];
 let mut next = Vec::with_capacity(cur.len().div_ceil(2));
 let mut i = 0;
 while i < cur.len() {
 let left = cur[i];
 let right = cur.get(i + 1).copied().unwrap_or(empties[level]);
 next.push(node_hash(&left, &right));
 i += 2;
 }
 if next.is_empty() {
 next.push(empties[level + 1]);
 }
 layers.push(next);
}
RegistryTree { layers, empties }
}

pub fn root(&self) -> [u8; 32] {
 self.layers[REGISTRY_DEPTH][0]
}

/// The authentication path (siblings, level 0 upward) for a leaf index.
pub fn path(&self, leaf_index: usize) -> Vec<[u8; 32]> {
 let mut path = Vec::with_capacity(REGISTRY_DEPTH);
 let mut idx = leaf_index;
 for level in 0..REGISTRY_DEPTH {
 let sib = idx ^ 1;
 let h = self.layers[level].get(sib).copied().unwrap_or(self.empties[level]);
 path.push(h);
 idx >>= 1;
 }
 path
}

#[cfg(test)]
mod tests {
 use super::*;

 fn leaf(d: &[u8]) -> [u8; 32] {
 registry_leaf(d, b"sel", b"modulus", b"\x01\x00\x01")
 }

 #[test]
 fn node_hash_is_sha256_of_concatenation() {
 // Pin the node primitive: SHA-256(left || right). Guards against a

```

```

// silent regression back to BN254 Poseidon (the 4× prove-time cause).
let left = [0x11u8; 32];
let right = [0x22u8; 32];
let mut expected = Sha256::new();
expected.update(left);
expected.update(right);
let expected: [u8; 32] = expected.finalize().into();
assert_eq!(node_hash(&left, &right), expected);
assert_ne!(node_hash(&left, &right), node_hash(&right, &left));
}

#[test]
fn leaf_is_deterministic_and_domain_sensitive() {
 assert_eq!(leaf(b"a.test"), leaf(b"a.test"));
 assert_ne!(leaf(b"a.test"), leaf(b"b.test"));
}

#[test]
fn leaf_sensitive_to_pubkey() {
 let a = registry_leaf(b"a.test", b"s", b"n1", b"\x01\x00\x01");
 let b = registry_leaf(b"a.test", b"s", b"n2", b"\x01\x00\x01");
 assert_ne!(a, b);
}

#[test]
fn build_then_verify_roundtrips_for_every_leaf() {
 let leaves: Vec<_> = (0u8..5).map(|i| leaf(&[b'd', i])).collect();
 let tree = RegistryTree::build(leaves.clone());
 let root = tree.root();
 for (i, l) in leaves.iter().enumerate() {
 let path = tree.path(i);
 assert_eq!(path.len(), REGISTRY_DEPTH);
 assert!(verify_membership(l, &path, i as u32, &root), "leaf {i} failed");
 }
}

#[test]
fn membership_rejects_wrong_root() {
 let leaves = vec![leaf(b"x"), leaf(b"y")];
 let tree = RegistryTree::build(leaves.clone());
 let mut bad = tree.root();
 bad[0] ^= 1;
 assert!(!verify_membership(&leaves[0], &tree.path(0), 0, &bad));
}

#[test]

```

```

fn membership_rejects_corrupted_sibling() {
 let leaves = vec![leaf(b"x"), leaf(b"y")];
 let tree = RegistryTree::build(leaves.clone());
 let mut path = tree.path(0);
 path[0][0] ^= 1;
 assert!(!verify_membership(&leaves[0], &path, 0, &tree.root()));
}

#[test]
fn membership_rejects_wrong_index() {
 let leaves = vec![leaf(b"x"), leaf(b"y"), leaf(b"z")];
 let tree = RegistryTree::build(leaves.clone());
 assert!(!verify_membership(&leaves[0], &tree.path(0), 1, &tree.root()));
}

#[test]
fn membership_rejects_non_member() {
 let leaves = vec![leaf(b"x"), leaf(b"y")];
 let tree = RegistryTree::build(leaves);
 let outsider = leaf(b"forger");
 // No valid path for a leaf that isn't in the tree.
 assert!(!verify_membership(&outsider, &tree.path(0), 0, &tree.root()));
}

#[test]
fn wrong_path_length_rejected() {
 let l = leaf(b"x");
 assert!(!verify_membership(&l, &[[0u8; 32]; 3], 0, &l));
}

#[test]
fn leaf_index_above_depth_rejected() {
 let leaves = vec![leaf(b"x"), leaf(b"y")];
 let tree = RegistryTree::build(leaves.clone());
 // Same low bits as a valid index 0, but a high bit set beyond DEPTH.
 let bad_index = 1u32 << REGISTRY_DEPTH;
 assert!(!verify_membership(&leaves[0], &tree.path(0), bad_index, &tree.root()));
}
}

...

methods/guest/src/dkim.rs

```rust
//! SPEC.md §4 steps 1 + 2 — locate the DKIM-Signature header in

```

```

///! `email_raw` and parse/validate its tag-value list.
///!
///! Hand-rolled per SPEC.md §9 "guest under 500 lines, hand-readable."
///! No external dep beyond `base64` for decoding `b=` and `bh=`.

use base64::{engine::general_purpose, Engine};

use crate::bytes_util::{
    bytes_eq_case_insensitive, is_wsp, is_wsp_or_crlf, strip_wsp_crlf, trim_wsp_crlf,
};

const HEADER_NAME: &[u8] = b"DKIM-Signature: ";

/// v0 supports only `relaxed/relaxed`. Other modes panic in step 2.
#[derive(Debug, PartialEq, Eq)]
pub enum Canonicalization {
    RelaxedRelaxed,
}

/// Validated DKIM-Signature header with required tags extracted and `b`/`bh`
/// base64-decoded.
#[derive(Debug)]
pub struct DkimHeader {
    pub canonicalization: Canonicalization,
    pub domain: Vec<u8>,
    pub selector: Vec<u8>,
    /// Raw bytes of the `h=` tag value (colon-separated list of header field
    /// names). Whitespace is preserved; parsing into individual names belongs
    /// to §4.5 (header set construction), not here.
    pub signed_headers_raw: Vec<u8>,
    /// Base64-decoded `bh=`.
    pub body_hash: Vec<u8>,
    /// Base64-decoded `b=` — the RSA signature.
    pub signature: Vec<u8>,
    /// Byte offset of `DKIM-Signature:` in `email_raw`.
    pub header_start: usize,
    /// Exclusive end of the full DKIM-Signature header (past the trailing
    /// CRLF that terminates it).
    pub header_end: usize,
}

/// SPEC.md §4 steps 1 + 2.
///
/// Locates the DKIM-Signature header at `offset`, bounds it (handling RFC
/// 5322 continuation lines), parses its tag-value list per RFC 6376 §3.2,
/// and applies every assertion enumerated in SPEC.md §4 step 2.

```

```

///
/// Panics (= guest aborts = no proof) on any assertion failure. Every panic
/// here is the intended behavior for an invalid prover input or a host that
/// lied about the witness.
pub fn locate_and_parse(
    email_raw: &[u8],
    offset: u32,
    claimed_domain: &[u8],
    witnessed_selector: &[u8],
    witnessed_signature_b64: &[u8],
    witnessed_body_hash_b64: &[u8],
) -> DkimHeader {
    let start = offset as usize;

    // §4.1: bounds + magic-byte check.
    assert!(
        start + HEADER_NAME.len() <= email_raw.len(),
        "dkim_header_index out of bounds"
    );
    assert!(
        bytes_eq_case_insensitive(&email_raw[start..start + HEADER_NAME.len()],
HEADER_NAME),
        "bytes at dkim_header_index are not 'DKIM-Signature:'"
    );

    // §4.1: find end of header.
    let value_start = start + HEADER_NAME.len();
    let header_end = find_header_end(email_raw, value_start);
    let value_end = if header_end >= 2 && &email_raw[header_end - 2..header_end] == b"\r\n" {
        header_end - 2
    } else {
        header_end
    };
    let header_value = &email_raw[value_start..value_end];

    // §4.2: parse tag-value list per RFC 6376 §3.2.
    let tags = parse_tag_list(header_value);

    let v = lookup(&tags, b"v").expect("v= missing");
    let a = lookup(&tags, b"a").expect("a= missing");
    let c = lookup(&tags, b"c").expect("c= missing");
    let d = lookup(&tags, b"d").expect("d= missing");
    let s = lookup(&tags, b"s").expect("s= missing");
    let h = lookup(&tags, b"h").expect("h= missing");
    let bh = lookup(&tags, b"bh").expect("bh= missing");
    let b_tag = lookup(&tags, b"b").expect("b= missing");

```

```

// SPEC.md §4 step 2: `l=` (body-length-limit) tag is not supported in v0.
// Accepting it without honoring it would let an attacker truncate the
// body-hash scope. Reject. Deferred to v1 (SPEC.md §8).
assert!(lookup(&tags, b"l").is_none(), "l= tag not supported in v0");

// §4.2: protocol version.
assert_eq!(trim_wsp_crlf(v), b"1", "v tag must be 1");

// §4.2: algorithm — v0 supports only rsa-sha256.
assert_eq!(trim_wsp_crlf(a), b"rsa-sha256", "a tag must be rsa-sha256");

// §4.2 ∩ §4.3 v0 restriction: relaxed/relaxed only.
assert_eq!(
    trim_wsp_crlf(c),
    b"relaxed/relaxed",
    "c tag must be relaxed/relaxed (v0)"
);

// §4.2 SOUNDNESS-CRITICAL: d == claimed_domain.
// This is the public-input binding probed by SPEC.md §7 adversarial #2.
// If this assertion is weakened or removed, the proof becomes vacuously
// trivial — a prover could claim any domain.
assert_eq!(
    trim_wsp_crlf(d),
    claimed_domain,
    "d tag does not match claimed_domain (public-input binding)"
);

// §4.2: s == witnessed_selector.
assert_eq!(trim_wsp_crlf(s), witnessed_selector, "s tag != witnessed selector");

// §4.2 / SPEC.md §5 host-trust guard: b and bh in the header must match
// what the host put in the witness. Compare after WSP/CRLF strip (RFC
// 6376 §3.5 allows whitespace within base64 values).
let b_clean = strip_wsp_crlf(b_tag);
let bh_clean = strip_wsp_crlf(bh);
let wb_clean = strip_wsp_crlf(witnessed_signature_b64);
let wbh_clean = strip_wsp_crlf(witnessed_body_hash_b64);

assert_eq!(b_clean, wb_clean, "b tag does not match witnessed signature");
assert_eq!(
    bh_clean, wbh_clean,
    "bh tag does not match witnessed body_hash"
);

```

```

// Decode base64.
let signature = general_purpose::STANDARD
    .decode(&b_clean)
    .expect("b= tag is not valid base64");
let body_hash = general_purpose::STANDARD
    .decode(&bh_clean)
    .expect("bh= tag is not valid base64");

DkimHeader {
    canonicalization: Canonicalization::RelaxedRelaxed,
    domain: trim_wsp_crlf(d).to_vec(),
    selector: trim_wsp_crlf(s).to_vec(),
    signed_headers_raw: trim_wsp_crlf(h).to_vec(),
    body_hash,
    signature,
    header_start: start,
    header_end,
}
}

// --- helpers -----

/// Returns exclusive byte offset just past the CRLF that ends the header.
/// RFC 5322: header ends at CRLF not followed by WSP, or EOF.
fn find_header_end(email_raw: &[u8], from: usize) -> usize {
    let mut i = from;
    while i + 1 < email_raw.len() {
        if email_raw[i] == b'r' && email_raw[i + 1] == b'n' {
            let next = i + 2;
            if next >= email_raw.len() || !is_wsp(email_raw[next]) {
                return next;
            }
        }
        i += 1;
    }
    email_raw.len()
}

/// RFC 6376 §3.2: tag-list = tag-spec *( ";" tag-spec ) [";"]
/// We split on `;` then on the first ` `. Tag values are NOT trimmed here
/// because some tags (h=, bh=, b=) carry whitespace that must be preserved
/// or specially stripped at use-site. Per-tag trimming is the caller's job.
fn parse_tag_list(input: &[u8]) -> Vec<(&[u8], &[u8])> {
    let mut tags = Vec::new();
    for piece in input.split(|&b| b == b';') {
        if piece.iter().all(|b| is_wsp_or_crlf(*b)) {

```



```

        continue;
    }
    let eq_idx = piece
        .iter()
        .position(|&b| b == b'=' )
        .expect("malformed DKIM tag (no '=')");
    let name = trim_wsp_crlf(&piece[..eq_idx]);
    let value = &piece[eq_idx + 1..];
    tags.push((name, value));
}
tags
}

fn lookup<'a>(tags: &'a [(&'a [u8], &'a [u8])], name: &[u8]) -> Option<&'a [u8]> {
    tags.iter().find(|(n, _)| *n == name).map(|(_, v)| *v)
}

// --- unit tests -----

#[cfg(test)]
mod tests {
    use super::*;

    // SHA-256 of empty input, base64 — a valid 32-byte body hash (44 chars + '=').
    const BH_VALID: &[u8] = b"47DEQpj8HBSa+/TImW+5JCeuQeRkm5NMpJWZG3hSuFU=";
    // 192 zero bytes encoded as base64 — exactly 256 chars, no padding needed
    // (192 % 3 == 0). Stand-in RSA signature; real RSA verify is §4.6.
    const B_VALID: &[u8] =

b"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAA\

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAA\

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAA\

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAA";

    fn header_bytes(
        v: &[u8],
        a: &[u8],
        c: &[u8],
        d: &[u8],

```

```

s: &[u8],
h: &[u8],
bh: &[u8],
b: &[u8],
) -> Vec<u8> {
    let mut out = Vec::from(b"DKIM-Signature: ".as_slice());
    for (k, val) in [
        (&b"v"[..], v), (b"a", a), (b"c", c), (b"d", d),
        (b"s", s), (b"h", h), (b"bh", bh), (b"b", b),
    ] {
        out.extend_from_slice(k);
        out.push(b'=');
        out.extend_from_slice(val);
        out.extend_from_slice(b"; ");
    }
    // Trim trailing "; " and end with CRLF.
    out.truncate(out.len() - 2);
    out.extend_from_slice(b"\r\n");
    out
}

fn default_header() -> Vec<u8> {
    header_bytes(
        b"1", b"rsa-sha256", b"relaxed/relaxed",
        b"example.com", b"selector1", b"From:To:Subject",
        BH_VALID, B_VALID,
    )
}

// -- happy path ---

#[test]
fn happy_path_valid_header_parses() {
    let header = default_header();
    let r = locate_and_parse(
        &header, 0, b"example.com", b"selector1", B_VALID, BH_VALID,
    );
    assert_eq!(r.domain, b"example.com");
    assert_eq!(r.selector, b"selector1");
    assert_eq!(r.signed_headers_raw, b"From:To:Subject");
    assert_eq!(r.canonicalization, Canonicalization::RelaxedRelaxed);
    assert_eq!(r.header_start, 0);
    assert!(r.header_end <= header.len());
    assert_eq!(r.body_hash.len(), 32); // SHA-256 output size
    assert!(r.signature.is_empty());
}

```

```

#[test]
fn case_insensitive_header_name() {
    let mut header = default_header();
    // Lowercase "DKIM-Signature:" prefix.
    for byte in &mut header[..15] {
        byte.make_ascii_lowercase();
    }
    let r = locate_and_parse(
        &header, 0, b"example.com", b"selector1", B_VALID, BH_VALID,
    );
    assert_eq!(r.domain, b"example.com");
}

#[test]
fn finds_header_end_before_next_header() {
    let mut block = default_header();
    block.extend_from_slice(b"Other-Header: bar\r\n");
    let r = locate_and_parse(
        &block, 0, b"example.com", b"selector1", B_VALID, BH_VALID,
    );
    // DKIM header should end exactly where Other-Header begins.
    assert_eq!(&block[r.header_end..r.header_end + 12], b"Other-Header");
}

#[test]
fn continuation_lines_in_header_value() {
    // Tags split across continuation lines (WSP after CRLF means same header).
    let header: Vec<u8> = format!(
        "DKIM-Signature: v=1; a=rsa-sha256;\r\n c=relaxed/relaxed; \
        d=example.com;\r\n\t s=selector1; h=From; bh={}; b={}\r\n",
        std::str::from_utf8(BH_VALID).unwrap(),
        std::str::from_utf8(B_VALID).unwrap(),
    )
    .into_bytes();
    let r = locate_and_parse(
        &header, 0, b"example.com", b"selector1", B_VALID, BH_VALID,
    );
    assert_eq!(r.domain, b"example.com");
}

// -- bounds + magic ---

#[test]
#[should_panic(expected = "dkim_header_index out of bounds")]
fn offset_past_end_panics() {

```

```

    locate_and_parse(b"short", 100, b"example.com", b"sel", B_VALID, BH_VALID);
}

#[test]
#[should_panic(expected = "not 'DKIM-Signature:')]
fn wrong_magic_panics() {
    locate_and_parse(
        b"Not-A-Header:  \r\n",
        0, b"example.com", b"sel", B_VALID, BH_VALID,
    );
}

// -- tag-value assertions ---

#[test]
#[should_panic(expected = "v tag must be 1")]
fn v_not_1_panics() {
    let header = header_bytes(
        b"2", b"rsa-sha256", b"relaxed/relaxed",
        b"example.com", b"selector1", b"From", BH_VALID, B_VALID,
    );
    locate_and_parse(&header, 0, b"example.com", b"selector1", B_VALID, BH_VALID);
}

#[test]
#[should_panic(expected = "a tag must be rsa-sha256")]
fn a_not_rsa_sha256_panics() {
    let header = header_bytes(
        b"1", b"rsa-sha1", b"relaxed/relaxed",
        b"example.com", b"selector1", b"From", BH_VALID, B_VALID,
    );
    locate_and_parse(&header, 0, b"example.com", b"selector1", B_VALID, BH_VALID);
}

#[test]
#[should_panic(expected = "c tag must be relaxed/relaxed")]
fn c_simple_simple_panics() {
    let header = header_bytes(
        b"1", b"rsa-sha256", b"simple/simple",
        b"example.com", b"selector1", b"From", BH_VALID, B_VALID,
    );
    locate_and_parse(&header, 0, b"example.com", b"selector1", B_VALID, BH_VALID);
}

// -- THE soundness-critical test (SPEC.md §7 adversarial #2) ---

```

```

#[test]
#[should_panic(expected = "public-input binding")]
fn d_mismatch_panics_SOUNDNESS_CRITICAL() {
    let header = default_header();
    // Prover passes a valid email signed by example.com but lies about
    // the claimed_domain public input.
    locate_and_parse(
        &header, 0, b"different.com", b"selector1", B_VALID, BH_VALID,
    );
}

#[test]
#[should_panic(expected = "witnessed selector")]
fn s_mismatch_panics() {
    let header = default_header();
    locate_and_parse(
        &header, 0, b"example.com", b"wrong-selector", B_VALID, BH_VALID,
    );
}

// -- host-trust guards ---

#[test]
#[should_panic(expected = "witnessed signature")]
fn b_mismatch_with_host_panics() {
    let header = default_header();
    locate_and_parse(
        &header, 0, b"example.com", b"selector1",
        b"AAAA", // host lies about what b= contains
        BH_VALID,
    );
}

#[test]
#[should_panic(expected = "witnessed body_hash")]
fn bh_mismatch_with_host_panics() {
    let header = default_header();
    locate_and_parse(
        &header, 0, b"example.com", b"selector1",
        B_VALID,
        b"AAAA",
    );
}

// -- v0 restrictions ---

```

```

#[test]
#[should_panic(expected = "l= tag not supported")]
fn l_tag_rejected() {
    let mut header = default_header();
    // Insert `; l=42` before the final CRLF.
    let cr_idx = header.len() - 2;
    let insertion = b"; l=42";
    header.splice(cr_idx..cr_idx, insertion.iter().copied());
    locate_and_parse(
        &header, 0, b"example.com", b"selector1", B_VALID, BH_VALID,
    );
}

```

```

#[test]
#[should_panic(expected = "b= missing")]
fn required_tag_missing_panics() {
    // Hand-construct a header without b=.
    let header: Vec<u8> = format!(
        "DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed; \
        d=example.com; s=selector1; h=From; bh={}\r\n",
        std::str::from_utf8(BH_VALID).unwrap(),
    ).into_bytes();
    locate_and_parse(
        &header, 0, b"example.com", b"selector1", B_VALID, BH_VALID,
    );
}

```

// -- base64 whitespace handling (RFC 6376 §3.5) ---

```

#[test]
fn whitespace_inside_base64_b_is_stripped_before_decode() {
    // Build a header with whitespace inserted in the middle of b=.
    let b_with_ws: Vec<u8> = {
        let mut v = Vec::new();
        for (i, ch) in B_VALID.iter().enumerate() {
            v.push(*ch);
            if i > 0 && i % 64 == 0 {
                v.extend_from_slice(b"\r\n\t");
            }
        }
        v
    };
    let header = header_bytes(
        b"1", b"rsa-sha256", b"relaxed/relaxed",
        b"example.com", b"selector1", b"From",
        BH_VALID, &b_with_ws,
    );
}

```

```

    );
    // Host gives clean witness; whitespace-stripped comparison should match.
    let r = locate_and_parse(
        &header, 0, b"example.com", b"selector1", B_VALID, BH_VALID,
    );
    assert(!r.signature.is_empty());
}
}

...

## methods/guest/src/canonical.rs

``rust
//! SPEC.md §4 step 3 — relaxed/relaxed canonicalization (RFC 6376 §3.4).
//!
//! Two pure functions:
//! - [`canonicalize_header_relaxed`] for RFC 6376 §3.4.2
//! - [`canonicalize_body_relaxed`] for RFC 6376 §3.4.4
//!
//! v0 supports only `relaxed/relaxed`. The caller (the guest pipeline) is
//! responsible for asserting `c=relaxed/relaxed` before invoking these
//! functions; this module assumes that check has passed.
//!
//! WSP per RFC 5234: SP (0x20) and HTAB (0x09). CRLF per RFC 5322: 0x0D 0x0A.
//! Shared byte helpers live in [`crate::bytes_util`].

use crate::bytes_util::{is_wsp, trim_wsp};

/// Apply RFC 6376 §3.4.2 "relaxed" header canonicalization to a single
/// header field, returning the canonicalized bytes including the final CRLF.
///
/// `name` is the field name (e.g. `b"Subject"`), without the trailing colon.
/// `value` is the field value, WITHOUT the trailing CRLF that terminates the
/// header in the original message. Internal CRLFs (continuation folding) MUST
/// remain in `value` and will be unfolded by this function.
///
/// RFC 6376 §3.4.2 algorithm, in order:
/// 1. Lowercase the field name.
/// 2. Unfold continuation lines (CRLF followed by WSP → WSP only).
/// 3. Collapse all WSP runs (across the now-unfolded value) to a single SP.
/// 4. Strip WSP at end of value.
/// 5. Strip WSP around the colon (i.e. strip leading WSP of the value;
///    the field name has no trailing WSP by construction).
///
/// Output format: `<lowercased name>`:`<canonicalized value>`\r\n`

```

```

pub fn canonicalize_header_relaxed(name: &[u8], value: &[u8]) -> Vec<u8> {
    let unfolded = unfold(value);
    let collapsed = collapse_wsp(&unfolded);
    let trimmed = trim_wsp(&collapsed);

    let mut out = Vec::with_capacity(name.len() + 1 + trimmed.len() + 2);
    for &b in name {
        out.push(b.to_ascii_lowercase());
    }
    out.push(b':');
    out.extend_from_slice(trimmed);
    out.extend_from_slice(b"\r\n");
    out
}

/// Apply RFC 6376 §3.4.4 "relaxed" body canonicalization.
///
/// Algorithm:
/// 1. For each line (split by CRLF): collapse WSP runs to a single SP, then
///    strip trailing WSP.
/// 2. Ignore all empty lines at the end of the message body.
/// 3. Terminate the result with exactly one CRLF.
///
/// The terminating CRLF is emitted unconditionally: per RFC 6376 §3.4.3
/// (referenced from §3.4.4 for empty-line semantics), an empty body
/// canonicalizes to a single CRLF; this matches what real DKIM signers
/// produce, e.g. `bh` for an empty body is SHA-256(`b"\r\n"`).
pub fn canonicalize_body_relaxed(body: &[u8]) -> Vec<u8> {
    // Split on CRLF. A trailing CRLF produces a final empty line which we'll
    // strip; a body with no trailing CRLF puts its last partial line in the
    // vec — that's fine, we still process it the same way.
    let mut lines: Vec<&[u8]> = Vec::new();
    let mut start = 0usize;
    let mut i = 0usize;
    while i + 1 < body.len() {
        if body[i] == b'\r' && body[i + 1] == b'\n' {
            lines.push(&body[start..i]);
            start = i + 2;
            i += 2;
        } else {
            i += 1;
        }
    }
    if start < body.len() {
        lines.push(&body[start..]);
    }
}

```



```

// Process each line.
let mut processed: Vec<Vec<u8>> = lines
    .into_iter()
    .map(|line| {
        let collapsed = collapse_wsp(line);
        strip_trailing_wsp(&collapsed).to_vec()
    })
    .collect();

// Strip trailing empty lines.
while processed.last().map_or(false, || l.is_empty()) {
    processed.pop();
}

// Rejoin with CRLF between lines, then unconditionally terminate with
// CRLF (RFC 6376 §3.4.3 inherited by §3.4.4 for the empty-body case).
let mut out = Vec::new();
for (idx, line) in processed.iter().enumerate() {
    if idx > 0 {
        out.extend_from_slice(b"\r\n");
    }
    out.extend_from_slice(line);
}
out.extend_from_slice(b"\r\n");
out
}

// --- internal helpers -----

/// Replace each `CRLF WSP` sequence with a single SP. Returns a new Vec.
/// (Subsequent WSP after the unfold boundary is left in place; the caller's
/// next step is collapse_wsp which deals with runs.)
fn unfold(input: &[u8]) -> Vec<u8> {
    let mut out = Vec::with_capacity(input.len());
    let mut i = 0;
    while i < input.len() {
        if i + 2 < input.len()
            && input[i] == b'\r'
            && input[i + 1] == b'\n'
            && is_wsp(input[i + 2])
        {
            // Drop CRLF, keep the following WSP byte verbatim.
            out.push(input[i + 2]);
            i += 3;
        } else {

```

```

        out.push(input[i]);
        i += 1;
    }
}
out
}

```

/// Collapse any run of WSP (SP or HTAB) to a single SP.

```

fn collapse_wsp(input: &[u8]) -> Vec<u8> {
    let mut out = Vec::with_capacity(input.len());
    let mut in_wsp = false;
    for &b in input {
        if is_wsp(b) {
            if !in_wsp {
                out.push(b' ');
                in_wsp = true;
            }
        } else {
            out.push(b);
            in_wsp = false;
        }
    }
    out
}

```

/// Strip trailing WSP only (canonical-specific; bytes_util has trim_wsp for
/// leading + trailing).

```

fn strip_trailing_wsp(input: &[u8]) -> &[u8] {
    let end = input
        .iter()
        .rposition(|&b| !is_wsp(b))
        .map(|i| i + 1)
        .unwrap_or(0);
    &input[..end]
}

```

// --- unit tests -----

```

#[cfg(test)]
mod tests {
    use super::*;

```

// -- header canonicalization --

```

#[test]
fn header_lowercases_field_name() {

```

```

    assert_eq!(
        canonicalize_header_relaxed(b"From", b"alice@example.com"),
        b"from:alice@example.com\r\n".to_vec()
    );
}

#[test]
fn header_preserves_value_case() {
    assert_eq!(
        canonicalize_header_relaxed(b"SUBJECT", b"AbC"),
        b"subject:AbC\r\n".to_vec()
    );
}

#[test]
fn header_collapses_internal_whitespace() {
    assert_eq!(
        canonicalize_header_relaxed(b"Subject", b"hello world\t\tfoo"),
        b"subject:hello world foo\r\n".to_vec()
    );
}

#[test]
fn header_strips_leading_wsp_after_colon() {
    // "From:  alice" → "from:alice"
    assert_eq!(
        canonicalize_header_relaxed(b"From", b"  alice@example.com"),
        b"from:alice@example.com\r\n".to_vec()
    );
}

#[test]
fn header_strips_trailing_wsp() {
    assert_eq!(
        canonicalize_header_relaxed(b"From", b"alice@example.com  \t"),
        b"from:alice@example.com\r\n".to_vec()
    );
}

#[test]
fn header_unfolds_continuation_line() {
    // Value with internal CRLF + WSP (a folded continuation line) must
    // unfold and collapse to a single SP between the parts.
    assert_eq!(
        canonicalize_header_relaxed(b"B", b"Y\r\n\tZ  "),
        b"b:Y Z\r\n".to_vec()
    );
}

```

```

    );
}

#[test]
fn header_unfolds_multiple_continuations() {
    assert_eq!(
        canonicalize_header_relaxed(b"H", b"a\r\n b\r\n c"),
        b"h:a b c\r\n".to_vec()
    );
}

#[test]
fn header_empty_value() {
    assert_eq!(
        canonicalize_header_relaxed(b"X-Empty", b""),
        b"x-empty:\r\n".to_vec()
    );
}

// -- body canonicalization --

#[test]
fn body_empty_canonicalizes_to_single_crlf() {
    // RFC 6376 §3.4.3 (inherited by §3.4.4): empty body → single CRLF.
    // Real DKIM signers produce bh = SHA-256(b"\r\n") for empty bodies.
    assert_eq!(canonicalize_body_relaxed(b""), b"\r\n".to_vec());
}

#[test]
fn body_only_crlfs_canonicalizes_to_single_crlf() {
    // All trailing empty lines stripped, then the unconditional CRLF
    // terminator is appended.
    assert_eq!(canonicalize_body_relaxed(b"\r\n\r\n\r\n"), b"\r\n".to_vec());
}

#[test]
fn body_single_line_gets_terminating_crlf() {
    assert_eq!(
        canonicalize_body_relaxed(b"Hello\r\n"),
        b"Hello\r\n".to_vec()
    );
}

#[test]
fn body_trailing_empty_lines_stripped() {
    assert_eq!(

```

```

        canonicalize_body_relaxed(b"Hello\r\n\r\n\r\n"),
        b"Hello\r\n".to_vec()
    );
}

#[test]
fn body_strips_trailing_wsp_per_line() {
    assert_eq!(
        canonicalize_body_relaxed(b"Hello \r\nWorld\t\r\n"),
        b"Hello\r\nWorld\r\n".to_vec()
    );
}

#[test]
fn body_collapses_internal_wsp() {
    assert_eq!(
        canonicalize_body_relaxed(b"a b\t\tc\r\n"),
        b"a b c\r\n".to_vec()
    );
}

#[test]
fn body_preserves_internal_empty_lines() {
    // An empty line BETWEEN non-empty lines is preserved; only trailing
    // empty lines are stripped.
    assert_eq!(
        canonicalize_body_relaxed(b"a\r\n\r\nb\r\n"),
        b"a\r\n\r\nb\r\n".to_vec()
    );
}

#[test]
fn body_no_trailing_crlf_still_terminated() {
    // RFC 6376 §3.4.4: canonicalized body always ends with CRLF if non-empty.
    // Defensive: if caller gave us a body without trailing CRLF, emit one.
    assert_eq!(
        canonicalize_body_relaxed(b"Hello"),
        b"Hello\r\n".to_vec()
    );
}

...

```

```
## methods/guest/src/body.rs
```

```

``rust
/// SPEC.md §4 step 4: SHA-256 the canonicalized body, assert equality with
/// `bh` from the DKIM-Signature header.
///
/// Also exposes `find_body_start` for callers that need to locate the
/// body region within an RFC 5322 message. The body begins immediately
/// after the first `\r\n\r\n` (the header/body separator).

use sha2::{Digest, Sha256};

use crate::canonical;

/// Returns the byte offset of the first byte of the body within `email`.
///
/// RFC 5322: headers and body are separated by an empty line, i.e. the
/// first occurrence of `\r\n\r\n`. Body starts at the byte immediately
/// after. If no separator is found, the message has no body and this
/// returns `email.len()`.
pub fn find_body_start(email: &[u8]) -> usize {
    let mut i = 0;
    while i + 3 < email.len() {
        if &email[i..i + 4] == b"\r\n\r\n" {
            return i + 4;
        }
        i += 1;
    }
    email.len()
}

/// SPEC.md §4 step 4.
///
/// Locates the body in `email`, applies relaxed body canonicalization
/// (RFC 6376 §3.4.4), SHA-256s the result, and asserts byte-equality
/// with `expected_body_hash`, which is the base64-decoded `bh` from
/// the §4.2 parser.
///
/// Panics (= guest aborts = no proof) on mismatch. SPEC.md §7 must-pass
/// #4 (`email_with_tampered_body_bh_mismatch_guest_panics`) is the test
/// that exercises this path.
pub fn verify_body_hash(email: &[u8], expected_body_hash: &[u8]) {
    let body_start = find_body_start(email);
    let body = &email[body_start..];
    let canonical_body = canonical::canonicalize_body_relaxed(body);
    let computed = Sha256::digest(&canonical_body);
    assert_eq!(
        computed.as_slice(),

```

```

        expected_body_hash,
        "computed body hash does not match bh"
    );
}

#[cfg(test)]
mod tests {
    use super::*;

    fn sha256(data: &[u8]) -> Vec<u8> {
        Sha256::digest(data).to_vec()
    }

    // -- find_body_start --

    #[test]
    fn find_body_start_after_separator() {
        let email = b"From: a@b\r\nSubject: x\r\n\r\nHello body\r\n";
        let start = find_body_start(email);
        assert_eq!(&email[start..], b"Hello body\r\n");
    }

    #[test]
    fn find_body_start_no_separator() {
        // No CRLF CRLF found, message is effectively all-headers, no body.
        let email = b"From: a@b\r\nSubject: x\r\n";
        assert_eq!(find_body_start(email), email.len());
    }

    #[test]
    fn find_body_start_empty_body_after_separator() {
        // Separator present, but nothing after it.
        let email = b"From: a@b\r\n\r\n";
        assert_eq!(find_body_start(email), email.len());
    }

    #[test]
    fn find_body_start_handles_separator_at_start() {
        // Pathological: no headers at all, message is just the separator
        // plus body.
        let email = b"\r\n\r\nBody";
        let start = find_body_start(email);
        assert_eq!(&email[start..], b"Body");
    }

    // -- verify_body_hash --

```

```

#[test]
fn verify_body_hash_matches_normal_body() {
    let email = b"From: a@b\r\n\r\nHello body\r\n";
    // The body "Hello body\r\n" canonicalizes to itself.
    let expected = sha256(b"Hello body\r\n");
    verify_body_hash(email, &expected);
}

#[test]
fn verify_body_hash_matches_empty_body() {
    // RFC 6376 §3.4.3/§3.4.4: empty body canonicalizes to b"\r\n".
    let email = b"From: a@b\r\n\r\n";
    let expected = sha256(b"\r\n");
    verify_body_hash(email, &expected);
}

#[test]
fn verify_body_hash_matches_canonicalized_body() {
    // Body has internal WSP that canonicalization collapses; `bh` must
    // reflect the canonicalized form, not the raw bytes.
    let email = b"From: a@b\r\n\r\nHello world\t\t!\r\n";
    let expected = sha256(b"Hello world !\r\n");
    verify_body_hash(email, &expected);
}

#[test]
#[should_panic(expected = "computed body hash does not match bh")]
fn verify_body_hash_panics_on_tampered_body() {
    // SPEC.md §7 must-pass #4 path.
    let email = b"From: a@b\r\n\r\nHello body\r\n";
    let wrong_bh = sha256(b"different content");
    verify_body_hash(email, &wrong_bh);
}

#[test]
#[should_panic(expected = "computed body hash does not match bh")]
fn verify_body_hash_panics_on_wrong_size_bh() {
    // Defensive: even an obviously-wrong-size bh must fail clearly,
    // not silently truncate or match a prefix.
    let email = b"From: a@b\r\n\r\nHello\r\n";
    verify_body_hash(email, b"too short");
}
}

...

```



```

## methods/guest/src/signed_set.rs

``rust
//! SPEC.md §4 step 5 — construct the canonicalized header set that gets
//! SHA-256'd and RSA-verified in §4.6.
//!
//! Per RFC 6376 §5.4:
//! 1. For each header field name in the `h=` tag (in the order they
//!    appear), find the corresponding header in the email above the
//!    DKIM-Signature, canonicalize it (relaxed), append to output.
//!    Multiple occurrences of the same name in `h=` consume successive
//!    occurrences top-to-bottom in the email (§5.4.2). A name in `h=`
//!    that does not match any header is treated as an empty value
//!    (§5.4 paragraph "Signers MAY claim ...").
//!
//! 2. Append the canonicalized DKIM-Signature header with the `b=` tag
//!    value emptied and NO trailing CRLF (§3.7 and §5.4 step 2).

use crate::body;
use crate::bytes_util::{bytes_eq_case_insensitive, strip_trailing_crlf, trim_wsp_crlf};
use crate::canonical;

/// SPEC.md §4 step 5. Returns the bytes that get SHA-256'd in §4.6.
pub fn build_signed_data(
    email: &[u8],
    h_tag_value: &[u8],
    dkim_header_start: usize,
    dkim_header_end: usize,
) -> Vec<u8> {
    let mut output = Vec::new();

    // RFC 6376 §5.4 says verifier consumes occurrences of each h= name
    // top-to-bottom across the FULL header block, treating missing
    // occurrences as null strings. In practice real-world DKIM signers
    // (Gmail tested here) and verifiers (dkimpy) instead search
    // bottom-to-top with a per-name lastindex cursor, and SKIP missing
    // occurrences rather than emitting empty `name:\r\n` placeholders.
    // Match real-world behavior so we can actually verify real emails;
    // the soundness story is unchanged because oversigning still works
    // (the signer just doesn't emit empty rows for missing names).
    let body_start = body::find_body_start(email);
    let headers = parse_headers(&email[..body_start]);

    // h= can be folded across lines, leaving CRLF + WSP inside a split piece.
    let names: Vec<&[u8]> = h_tag_value

```

```

.split(|&c| c == b':')
.map(trim_wsp_crlf)
.filter(|n| !n.is_empty())
.collect();

// dkimpy-style: for each name in h=, scan headers from `lastindex[name]`
// (default = headers.len()) downward, find the first match, append, and
// update lastindex to the matched position.
let mut last_index: Vec<(Vec<u8>, usize)> = Vec::new();
for include_name in &names {
    let lc: Vec<u8> = include_name.iter().map(|b| b.to_ascii_lowercase()).collect();
    let mut i = last_index
        .iter()
        .find(|(k, _)| *k == lc)
        .map(|(_, v)| *v)
        .unwrap_or(headers.len());
    let mut matched: Option<usize> = None;
    while i > 0 {
        i -= 1;
        if bytes_eq_case_insensitive(headers[i].name, include_name) {
            matched = Some(i);
            break;
        }
    }
    if let Some(idx) = matched {
        let h = &headers[idx];
        let canonicalized = canonical::canonicalize_header_relaxed(h.name, h.value);
        output.extend_from_slice(&canonicalized);
        if let Some(entry) = last_index.iter_mut().find(|(k, _)| *k == lc) {
            entry.1 = idx;
        } else {
            last_index.push((lc, idx));
        }
    }
    // If no match, skip (NOT emit empty). Matches dkimpy and real signers.
}

// Append the DKIM-Signature header with b= emptied, no trailing CRLF.
let dkim_header = &email[dkim_header_start..dkim_header_end];
let (name, value) = split_header_name_value(dkim_header);
let canonicalized = canonical::canonicalize_header_relaxed(name, value);
let b_emptied = null_b_tag(&canonicalized);
let no_crlf = strip_trailing_crlf(&b_emptied);
output.extend_from_slice(no_crlf);

output

```

```

}

/// v1 design §4.9. True if `name` (lowercase) is one of the header field
/// names listed in the `h=` tag value, i.e. the header is covered by the
/// DKIM signature. Case-insensitive; tolerates folding WSP/CRLF in `h=`.
pub fn h_contains(h_tag_value: &[u8], name: &[u8]) -> bool {
    h_tag_value
        .split(|&c| c == b':')
        .map(trim_wsp_crlf)
        .filter(|n| !n.is_empty())
        .any(|n| bytes_eq_case_insensitive(n, name))
}

/// v1 design §4.9. Return the raw value bytes of the bottom-most header
/// in the message header block matching `name` (case-insensitive). For a
/// header listed once in `h=`, that bottom-most occurrence is exactly the
/// instance the signature covers (RFC 6376 §5.4.2 bottom-up selection, as
/// implemented in `build_signed_data`). Reading the signed instance — never
/// an unsigned prepended duplicate — is what makes disclosure header-prepend
/// safe: a `From:` prepended above the signed one is not returned here, and a
/// `From:` appended below it would change the signed data and fail RSA verify.
///
/// Returns `None` if no header with that name exists in the block.
///
/// Parses the entire header block (same as `build_signed_data`), so the
/// instance returned here is byte-identical to the one `build_signed_data`
/// fed to RSA for this name's first `h=` mention. That consistency is what
/// guarantees we disclose exactly the signed bytes.
pub fn signed_header_value(email: &[u8], name: &[u8]) -> Option<Vec<u8>> {
    let body_start = body::find_body_start(email);
    let headers = parse_headers(&email[..body_start]);
    headers
        .iter()
        .rev()
        .find(|h| bytes_eq_case_insensitive(h.name, name))
        .map(|h| h.value.to_vec())
}

// -- helpers -----

/// One parsed header from the message header block: name (case preserved)
/// and value (no leading WSP unstripped, no trailing CRLF; continuation
/// CRLF+WSP preserved so the canonicalizer unfolds them).
struct ParsedHeader<'a> {
    name: &'a [u8],
    value: &'a [u8],

```

```
}
```

```
/// Parse the header block into individual headers in message order.
```

```
fn parse_headers(headers_block: &[u8]) -> Vec<ParsedHeader<'_>> {  
    let mut out = Vec::new();  
    let mut i = 0;  
    while i < headers_block.len() {  
        let line_end = find_field_end(headers_block, i);  
        if let Some(c) = headers_block[i..line_end].iter().position(|&b| b == b':') {  
            let name = &headers_block[i..i + c];  
            let value_start = i + c + 1;  
            let value_end = if line_end >= 2 && &headers_block[line_end - 2..line_end] == b"\r\n" {  
                line_end - 2  
            } else {  
                line_end  
            };  
            out.push(ParsedHeader {  
                name,  
                value: &headers_block[value_start..value_end],  
            });  
            i = line_end;  
        }  
    }  
    out  
}
```

```
/// RFC 5322 header bound: end of header = CRLF not followed by WSP, or EOF.
```

```
fn find_field_end(block: &[u8], from: usize) -> usize {  
    let mut i = from;  
    while i + 1 < block.len() {  
        if block[i] == b'\r' && block[i + 1] == b'\n' {  
            let next = i + 2;  
            if next >= block.len() || (block[next] != b' ' && block[next] != b'\t') {  
                return next;  
            }  
        }  
        i += 1;  
    }  
    block.len()  
}
```

```
/// Split a single complete header (name:value...CRLF) into name and value.
```

```
fn split_header_name_value(header: &[u8]) -> (&[u8], &[u8]) {  
    let colon = header.iter().position(|&c| c == b':').expect("header has no colon");  
    let name = &header[..colon];  
    let value_end = if header.len() >= 2 && &header[header.len() - 2..] == b"\r\n" {
```

```

        header.len() - 2
    } else {
        header.len()
    };
    let value = &header[colon + 1..value_end];
    (name, value)
}

/// Given a canonicalized header (name:value\r\n or name:value), produce a
/// copy with the `b=` tag value emptied. Used per RFC 6376 §3.7 to compute
/// the DKIM-Signature contribution to the signed data.
fn null_b_tag(canonicalized: &[u8]) -> Vec<u8> {
    let colon = canonicalized.iter().position(|&c| c == b':').expect("no colon");
    let mut out = Vec::with_capacity(canonicalized.len());
    out.extend_from_slice(&canonicalized[..=colon]);

    let value_part = &canonicalized[colon + 1..];
    let (vp, trailing) = if value_part.ends_with(b"\r\n") {
        (&value_part[..value_part.len() - 2], &b"\r\n"[..])
    } else {
        (value_part, &b""[..])
    };

    let pieces: Vec<&[u8]> = vp.split(|&c| c == b':').collect();
    for (idx, piece) in pieces.iter().enumerate() {
        if idx > 0 {
            out.push(b';');
        }
        if let Some(eq) = piece.iter().position(|&c| c == b'=') {
            let tag_name = trim_wsp_crlf(&piece[..eq]);
            if tag_name == b"b" {
                out.extend_from_slice(&piece[..=eq]);
                // value dropped
            } else {
                out.extend_from_slice(piece);
            }
        } else {
            out.extend_from_slice(piece);
        }
    }
    out.extend_from_slice(trailing);
    out
}

// -- unit tests -----

```

```

#[cfg(test)]
mod tests {
    use super::*;

    /// Builds (email_bytes, dkim_start, dkim_end) for tests.
    fn make_email(headers_before: &[u8], dkim: &[u8], body: &[u8]) -> (Vec<u8>, usize, usize) {
        let mut email = Vec::new();
        email.extend_from_slice(headers_before);
        let dkim_start = email.len();
        email.extend_from_slice(dkim);
        let dkim_end = email.len();
        email.extend_from_slice(b"\r\n");
        email.extend_from_slice(body);
        (email, dkim_start, dkim_end)
    }

    #[test]
    fn single_header_canonicalized_and_appended() {
        let (email, ds, de) = make_email(
            b"From: alice@example.com\r\n",
            b"DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed; \
            d=example.com; s=sel; h=From; bh=xx; b=signature\r\n",
            b"body",
        );
        let out = build_signed_data(&email, b"From", ds, de);
        let expected = b"from:alice@example.com\r\n\
            dkim-signature:v=1; a=rsa-sha256; c=relaxed/relaxed; \
d=example.com; s=sel; h=From; bh=xx; b=";
        assert_eq!(&out[..], &expected[..]);
    }

    #[test]
    fn multiple_headers_in_h_tag() {
        let (email, ds, de) = make_email(
            b"From: a@b\r\nTo: c@d\r\nSubject: hi\r\n",
            b"DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed; \
            d=example.com; s=sel; h=From:To:Subject; bh=xx; b=sig\r\n",
            b"body",
        );
        let out = build_signed_data(&email, b"From:To:Subject", ds, de);
        // Each header canonicalized in h= order, then DKIM-Signature with b= empty no CRLF.
        let s = std::str::from_utf8(&out).unwrap();
        assert!(s.starts_with("from:a@b\r\nto:c@d\r\nsubject:hi\r\n"));
        assert!(s.ends_with("b="));
        assert!(s.contains("dkim-signature:"));
    }
}

```

```

#[test]
fn missing_header_in_h_tag_is_skipped() {
    // dkimpy-style: a name in h= that doesn't match any header is SKIPPED,
    // not emitted as empty. Matches real-world signers (Gmail, etc.).
    // RFC 6376 §5.4 text says "treat as null string" but the practical
    // convention diverges.
    let (email, ds, de) = make_email(
        b"From: a@b\r\n",
        b"DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed; \
        d=example.com; s=sel; h=From:Date; bh=xx; b=sig\r\n",
        b"body",
    );
    let out = build_signed_data(&email, b"From:Date", ds, de);
    let s = std::str::from_utf8(&out).unwrap();
    // "from:a@b\r\n" then directly DKIM-Signature (no "date:\r\n" inserted).
    assert!(s.starts_with("from:a@b\r\ndkim-signature:"), "got: {:?}", &s[..s.len().min(60)]);
}

```

```

#[test]
fn multiple_occurrences_bottom_to_top() {
    // dkimpy convention: with two From headers and h=From:From, the FIRST
    // mention in h= picks the LAST occurrence in the message (bottom), the
    // SECOND mention picks the one above it. Matches dkimpy's select_headers.
    let (email, ds, de) = make_email(
        b"From: first@x\r\nFrom: second@y\r\n",
        b"DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed; \
        d=example.com; s=sel; h=From:From; bh=xx; b=sig\r\n",
        b"body",
    );
    let out = build_signed_data(&email, b"From:From", ds, de);
    let s = std::str::from_utf8(&out).unwrap();
    assert!(s.starts_with("from:second@y\r\nfrom:first@x\r\n"));
}

```

```

#[test]
fn case_insensitive_header_name_match() {
    // The email header is "from:" (lowercase), the h= value names "FROM"
    // (uppercase). Matching is case-insensitive, and relaxed
    // canonicalization always lowercases the field name in the output.
    let (email, ds, de) = make_email(
        b"from: lowercase@x\r\n",
        b"DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed; \
        d=example.com; s=sel; h=FROM; bh=xx; b=sig\r\n",
        b"body",
    );
}

```

```

    let out = build_signed_data(&email, b"FROM", ds, de);
    let s = std::str::from_utf8(&out).unwrap();
    assert!(s.starts_with("from:lowercase@x\r\n"));
}

#[test]
fn null_b_tag_with_b_at_end() {
    // canonicalized: "dkim-signature:v=1; b=abc"
    let inp = b"dkim-signature:v=1; b=abc\r\n";
    let out = null_b_tag(inp);
    assert_eq!(out, b"dkim-signature:v=1; b=\r\n");
}

#[test]
fn null_b_tag_with_b_in_middle() {
    let inp = b"dkim-signature:v=1; b=abc; bh=xyz\r\n";
    let out = null_b_tag(inp);
    assert_eq!(out, b"dkim-signature:v=1; b=; bh=xyz\r\n");
}

#[test]
fn null_b_tag_with_b_first_tag() {
    let inp = b"dkim-signature:b=abc; v=1\r\n";
    let out = null_b_tag(inp);
    assert_eq!(out, b"dkim-signature:b=; v=1\r\n");
}

#[test]
fn null_b_tag_does_not_touch_bh_tag() {
    // The tag name 'bh' shares a prefix with 'b' but is a different tag.
    let inp = b"dkim-signature:v=1; bh=xyz; b=abc\r\n";
    let out = null_b_tag(inp);
    assert_eq!(out, b"dkim-signature:v=1; bh=xyz; b=\r\n");
}

#[test]
fn parse_headers_extracts_name_and_value() {
    let block = b"From: alice@x\r\nTo: bob@y\r\n";
    let h = parse_headers(block);
    assert_eq!(h.len(), 2);
    assert_eq!(h[0].name, b"From");
    assert_eq!(h[0].value, b" alice@x");
    assert_eq!(h[1].name, b"To");
    assert_eq!(h[1].value, b" bob@y");
}

```



```

#[test]
fn h_contains_matches_case_insensitively_and_tolerates_wsp() {
    assert!(h_contains(b"from : to : subject : date", b"from"));
    assert!(h_contains(b"From:To:Subject", b"to"));
    assert!(!h_contains(b"to:subject:date", b"from"));
}

#[test]
fn signed_header_value_returns_bottom_most_instance() {
    // Two From headers (a prepended attacker line above the real one).
    // Bottom-most = the signed instance.
    let email = b"From: attacker@evil.test\r\n\
        From: real@corp.test\r\n\
        To: ops@corp.test\r\n\
        \r\nbody";
    assert_eq!(
        signed_header_value(email, b"from").unwrap(),
        b" real@corp.test".to_vec()
    );
    assert_eq!(
        signed_header_value(email, b"to").unwrap(),
        b" ops@corp.test".to_vec()
    );
    assert!(signed_header_value(email, b"subject").is_none());
}

#[test]
fn output_ends_without_crlf() {
    // The DKIM-Signature canonicalized chunk MUST NOT end with CRLF
    // (RFC 6376 §3.7 / §5.4 step 2).
    let (email, ds, de) = make_email(
        b"From: a@b\r\n",
        b"DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed; \
            d=example.com; s=sel; h=From; bh=xx; b=sig\r\n",
        b"body",
    );
    let out = build_signed_data(&email, b"From", ds, de);
    assert!(!out.ends_with(b"\r\n"), "signed data must not end with CRLF");
}

...

## methods/guest/src/verify.rs

```rust

```

```
/// SPEC.md §4 step 6 — RSA-PKCS1#v1.5-SHA256 verify of the DKIM signature
/// over the signed data produced in §4.5.
///
/// This is the expensive step in the guest. RISC0's bigint accelerator
/// syscalls are reached transitively via `num-bigint` (used by `rsa`);
/// adopting the upstream RISC0 patches for `rsa`/`num-bigint` is a v1
/// prove-time optimization, not done in v0.
```

```
use rsa::{pkcs1v15::Pkcs1v15Sign, BigUint, RsaPublicKey};
use sha2::{Digest, Sha256};
```

```
/// SPEC.md §4 step 6.
///
/// SHA-256s `signed_data`, then PKCS#1 v1.5 RSA-verifies `signature`
/// against it using the public key (modulus `n`, exponent `e`, both
/// supplied as big-endian byte slices via the `PublicInputs`).
///
/// Panics (= guest aborts = no proof) on any failure: invalid pubkey
/// encoding, signature/hash size mismatch, or cryptographic verification
/// failure. The §7 must-pass #5 and adversarial bit-flip tests exercise
/// these failure paths against a real RSA keypair built in the host.
pub fn verify_rsa_signature(
 signed_data: &[u8],
 signature: &[u8],
 pubkey_n: &[u8],
 pubkey_e: &[u8],
) {
 let n = BigUint::from_bytes_be(pubkey_n);
 let e = BigUint::from_bytes_be(pubkey_e);
 let pubkey = RsaPublicKey::new(n, e).expect("invalid RSA public key");
 let hashed = Sha256::digest(signed_data);
 pubkey
 .verify(Pkcs1v15Sign::new::(<Sha256>(), &hashed, signature)
 .expect("RSA signature verification failed");
}
```

```
#[cfg(test)]
mod tests {
 use super::*;
 use rand::rngs::OsRng;
 use rsa::traits::PublicKeyParts;
 use rsa::{RsaPrivateKey, RsaPublicKey};
 use std::sync::OnceLock;
```

```
/// Generate (private, public) once per test process. RSA-2048 keygen
/// is ~1-2s; cache it so all tests share one keypair.
```

```

fn keypair() -> &'static (RsaPrivateKey, RsaPublicKey) {
 static KP: OnceLock<(RsaPrivateKey, RsaPublicKey)> = OnceLock::new();
 KP.get_or_init(|| {
 let mut rng = OsRng;
 let priv_key = RsaPrivateKey::new(&mut rng, 2048).expect("keygen");
 let pub_key = RsaPublicKey::from(&priv_key);
 (priv_key, pub_key)
 })
}

fn pubkey_bytes(pk: &RsaPublicKey) -> (Vec<u8>, Vec<u8>) {
 (pk.n().to_bytes_be(), pk.e().to_bytes_be())
}

fn sign(priv_key: &RsaPrivateKey, signed_data: &[u8]) -> Vec<u8> {
 let hashed = Sha256::digest(signed_data);
 priv_key
 .sign(Pkcs1v15Sign::new:::<Sha256>(), &hashed)
 .expect("sign")
}

#[test]
fn happy_path_valid_signature_verifies() {
 let (priv_key, pub_key) = keypair();
 let signed_data = b"any bytes that the signer signed";
 let signature = sign(priv_key, signed_data);
 let (n, e) = pubkey_bytes(pub_key);
 verify_rsa_signature(signed_data, &signature, &n, &e);
}

#[test]
#[should_panic(expected = "RSA signature verification failed")]
fn bit_flip_in_signature_panics() {
 // SPEC.md §7 soundness sanity check.
 let (priv_key, pub_key) = keypair();
 let signed_data = b"any bytes that the signer signed";
 let mut signature = sign(priv_key, signed_data);
 signature[0] ^= 0x01;
 let (n, e) = pubkey_bytes(pub_key);
 verify_rsa_signature(signed_data, &signature, &n, &e);
}

#[test]
#[should_panic(expected = "RSA signature verification failed")]
fn tampered_signed_data_panics() {
 // SPEC.md §7 must-pass #5: tampered signed data => verify fails.

```

```

 let (priv_key, pub_key) = keypair();
 let signed_data = b"any bytes that the signer signed";
 let signature = sign(priv_key, signed_data);
 let (n, e) = pubkey_bytes(pub_key);
 let tampered = b"DIFFERENT bytes than the signer signed";
 verify_rsa_signature(tampered, &signature, &n, &e);
}

#[test]
#[should_panic(expected = "RSA signature verification failed")]
fn wrong_pubkey_panics() {
 let (priv_key, _) = keypair();
 let signed_data = b"any bytes";
 let signature = sign(priv_key, signed_data);
 // Build an unrelated keypair and verify against THAT pubkey.
 let mut rng = OsRng;
 let other_priv = RsaPrivateKey::new(&mut rng, 2048).expect("keygen");
 let other_pub = RsaPublicKey::from(&other_priv);
 let (n, e) = pubkey_bytes(&other_pub);
 verify_rsa_signature(signed_data, &signature, &n, &e);
}
}

...

methods/guest/src/nullifier.rs

``rust
/// SPEC.md §4 step 7 — Poseidon-based replay nullifier.
///
/// `nullifier = Poseidon(domain_separator_v0, claimed_domain, signature)`,
/// where each of the three byte-string inputs is compressed to a single
/// BN254 field element via SHA-256 (then reduced mod the BN254 prime),
/// and Poseidon is the Circom/Iden3-parameterized BN254 instance via
/// the `light-poseidon` crate.
///
/// Per SPEC.md §3 the output is a 32-byte field element written into the
/// `PublicOutputs::nullifier` slot.

use ark_bn254::Fr;
use ark_ff::{BigInteger, PrimeField};
use light_poseidon::{Poseidon, PoseidonHasher};
use sha2::{Digest, Sha256};

/// SPEC.md §4 step 7.
pub fn compute_nullifier(

```

```

 domain_separator_v0: &[u8],
 claimed_domain: &[u8],
 signature: &[u8],
) -> [u8; 32] {
 let f1 = bytes_to_field(domain_separator_v0);
 let f2 = bytes_to_field(claimed_domain);
 let f3 = bytes_to_field(signature);

 let mut hasher = Poseidon::::new_circom(3).expect("poseidon init for t=4");
 let result = hasher.hash(&[f1, f2, f3]).expect("poseidon hash failed");

 let be_bytes = result.into_bigint().to_bytes_be();
 let mut out = [0u8; 32];
 let start = 32 - be_bytes.len();
 out[start..].copy_from_slice(&be_bytes);
 out
}

/// Compress arbitrary-length bytes to one BN254 field element.
///
/// SHA-256 first (one-way + uniform 256-bit digest), then reduce mod the
/// BN254 base-field prime via `from_be_bytes_mod_order`. The reduction is
/// slightly biased over the top ~3 bits, but for nullifier purposes
/// (collision resistance against an adversary who must also forge a valid
/// DKIM RSA signature) the bias is far below cryptographic concern.
fn bytes_to_field(bytes: &[u8]) -> Fr {
 let digest = Sha256::digest(bytes);
 Fr::from_be_bytes_mod_order(&digest)
}

#[cfg(test)]
mod tests {
 use super::*;

 const DOM: &[u8] = b"0nce-v0-nullifier";

 #[test]
 fn nullifier_is_deterministic() {
 let a = compute_nullifier(DOM, b"example.com", b"signature-bytes");
 let b = compute_nullifier(DOM, b"example.com", b"signature-bytes");
 assert_eq!(a, b);
 }

 #[test]
 fn different_signature_yields_different_nullifier() {
 let a = compute_nullifier(DOM, b"example.com", b"sig-a");

```

```

 let b = compute_nullifier(DOM, b"example.com", b"sig-b");
 assert_ne!(a, b);
}

#[test]
fn different_domain_yields_different_nullifier() {
 let a = compute_nullifier(DOM, b"example.com", b"sig");
 let b = compute_nullifier(DOM, b"corp.example", b"sig");
 assert_ne!(a, b);
}

#[test]
fn different_domain_separator_yields_different_nullifier() {
 // If we ever rev to v1 and change domain_separator, every prior
 // proof's nullifier must be different. This is what makes the
 // separator constant load-bearing: changing it invalidates the
 // entire prior nullifier corpus.
 let a = compute_nullifier(b"0nce-v0-nullifier", b"example.com", b"sig");
 let b = compute_nullifier(b"0nce-v1-nullifier", b"example.com", b"sig");
 assert_ne!(a, b);
}

#[test]
fn output_is_32_bytes() {
 let n = compute_nullifier(DOM, b"d", b"s");
 assert_eq!(n.len(), 32);
}
}

...

methods/guest/src/address.rs

``rust
//! v1 design §4.10 — parse the disclosed identity header's email address.
//!
//! Restricted grammar (v1 only; assert-and-fail otherwise, mirroring v0's
//! relaxed/relaxed-only discipline):
//!
//! header-value = [display-name] "<" addr-spec ">" / addr-spec
//! addr-spec = local-part "@" domain
//! local-part = dot-atom ; ASCII atext + "."
//! domain = dot-atom of LDH labels
//!
//! No quoted strings, comments, groups, multiple addresses, or folding
//! *inside* the addr-spec. A `display-name <addr>` wrapper IS allowed.

```

```

//!
//! `extract_address` panics (= guest aborts = no proof) on anything outside
//! the grammar. Every panic is intended: a malformed address must not yield
//! a proof.
//!
//! Note (v1 design refinement): the guest does NOT assert that the address
//! domain equals `claimed_domain`. Real-world mail is frequently signed by a
//! provider whose `d=` differs from the From domain (Gmail/Workspace sign
//! `d=google.com` over `From: you@yourcompany.com`). Whether the disclosed
//! address aligns with the signing domain is a *policy* decision the verifier
//! makes from the two public values (`claimed_domain`, `disclosed_address`);
//! see the host verifier. The soundness guarantee — that the address comes
//! from the DKIM-signed header set, not a forged one — lives in §4.9.

/// Parse the disclosed header value and return the canonical `local@domain`
/// bytes with the domain lowercased (local-part case preserved, per
/// RFC 5321 §2.4).
pub fn extract_address(header_value: &[u8]) -> Vec<u8> {
 // Unfold: drop CR/LF so folded values become single-line. Internal WSP
 // is preserved here; the addr-spec extraction below handles it.
 let unfolded: Vec<u8> = header_value
 .iter()
 .copied()
 .filter(|&b| b != b'\r' && b != b'\n')
 .collect();
 let trimmed = trim_wsp(&unfolded);

 // Extract the addr-spec: either inside the last `<...>`, or the whole
 // token when there is no angle-bracket wrapper.
 let addr = if let Some(lt) = trimmed.iter().position(|&b| b == b'<') {
 let gt = trimmed
 .iter()
 .rposition(|&b| b == b'>')
 .expect("address has '<' without matching '>'");
 assert!(gt > lt, "malformed angle-bracket address");
 trim_wsp(&trimmed[lt + 1..gt])
 } else {
 trimmed
 };

 // A bare addr-spec must contain no internal whitespace.
 assert!(
 !addr.iter().any(|&b| b == b' ' || b == b'\t'),
 "addr-spec contains whitespace"
);
}

```

```

// Split on the single '@'.
let at = addr.iter().position(|&b| b == b'@').expect("addr-spec has no '@");
assert!(
 !addr[at + 1..].iter().any(|&b| b == b'@'),
 "addr-spec has more than one '@"
);
let local = &addr[..at];
let domain = &addr[at + 1..];

assert!(is_dot_atom_local(local), "invalid local-part");
assert!(is_ldh_domain(domain), "invalid domain");

let domain_lower: Vec<u8> = domain.iter().map(|b| b.to_ascii_lowercase()).collect();

let mut out = Vec::with_capacity(local.len() + 1 + domain_lower.len());
out.extend_from_slice(local);
out.push(b'@');
out.extend_from_slice(&domain_lower);
out
}

fn trim_wsp(b: &[u8]) -> &[u8] {
 let mut start = 0;
 let mut end = b.len();
 while start < end && (b[start] == b' ' || b[start] == b'\t') {
 start += 1;
 }
 while end > start && (b[end - 1] == b' ' || b[end - 1] == b'\t') {
 end -= 1;
 }
 &b[start..end]
}

/// RFC 5322 atext plus '.', with no leading/trailing/consecutive dots.
fn is_dot_atom_local(s: &[u8]) -> bool {
 if s.is_empty() || s[0] == b'.' || s[s.len() - 1] == b'.' {
 return false;
 }
 let mut prev_dot = false;
 for &c in s {
 if c == b'.' {
 if prev_dot {
 return false;
 }
 prev_dot = true;
 continue;
 }
 }
}

```



```

 }
 prev_dot = false;
 if !is_atext(c) {
 return false;
 }
}
true
}

fn is_atext(c: u8) -> bool {
 c.is_ascii_alphanumeric()
 || matches!(
 c,
 b'!' | b'#' | b'$' | b'%' | b'&' | b\" | b'*' | b'+' | b'/' | b'='
 | b'?' | b'^' | b'_' | b'`' | b'{' | b'|' | b'}' | b'~' | b'-'
)
}

/// Dot-separated LDH (letter/digit/hyphen) labels; each label non-empty and
/// not starting or ending with a hyphen.
fn is_ldh_domain(s: &[u8]) -> bool {
 if s.is_empty() {
 return false;
 }
 for label in s.split(|&b| b == b'.') {
 if label.is_empty() || label[0] == b'-' || label[label.len() - 1] == b'-' {
 return false;
 }
 if !label.iter().all(|&c| c.is_ascii_alphanumeric() || c == b'-') {
 return false;
 }
 }
}
true
}

#[cfg(test)]
mod tests {
 use super::*;

 #[test]
 fn bare_addr_spec() {
 assert_eq!(extract_address(b" alice@example.com"), b"alice@example.com");
 }

 #[test]
 fn display_name_wrapped() {

```

```

 assert_eq!(
 extract_address(b" Display Name <bob@example.com>"),
 b"bob@example.com"
);
}

#[test]
fn domain_lowercased_local_preserved() {
 assert_eq!(
 extract_address(b"Bob.Smith <Bob.Smith@Example.COM>"),
 b"Bob.Smith@example.com"
);
}

#[test]
fn folded_value_unfolded() {
 assert_eq!(
 extract_address(b" Name\r\n <a@sub.example.com>"),
 b"a@sub.example.com"
);
}

#[test]
#[should_panic(expected = "no '@'")]
fn no_at_sign_panics() {
 extract_address(b"not-an-address");
}

#[test]
#[should_panic(expected = "more than one '@'")]
fn double_at_panics() {
 extract_address(b"a@b@example.com");
}

#[test]
#[should_panic(expected = "whitespace")]
fn bare_internal_space_panics() {
 extract_address(b"no at sign here@example.com");
}

#[test]
#[should_panic(expected = "invalid local-part")]
fn empty_local_panics() {
 extract_address(b"@example.com");
}

```

```

#[test]
#[should_panic(expected = "without matching '>')]
fn open_angle_without_close_panics() {
 extract_address(b"Name <a@example.com");
}

#[test]
#[should_panic(expected = "invalid local-part")]
fn leading_dot_local_panics() {
 extract_address(b".alice@example.com");
}
}

...

methods/guest/src/bytes_util.rs

```rust
/// Byte-slice helpers shared by the parsing / canonicalization modules.
/// Consolidated here so the trim/strip logic exists in exactly one place
/// and can't drift between modules (the failure mode would be
/// "canonicalization disagrees with DKIM parse," which produces a
/// valid-looking proof that doesn't actually prove SPEC.md §2).

/// SP (0x20) or HTAB (0x09).
#[inline]
pub fn is_wsp(b: u8) -> bool {
    b == b' ' || b == b'\t'
}

/// SP, HTAB, CR, or LF.
#[inline]
pub fn is_wsp_or_crlf(b: u8) -> bool {
    is_wsp(b) || b == b'\r' || b == b'\n'
}

/// Strip leading and trailing SP/HTAB only. CRLF is left intact.
pub fn trim_wsp(input: &[u8]) -> &[u8] {
    let start = input.iter().position(|&b| !is_wsp(b)).unwrap_or(input.len());
    let end = input.iter().rposition(|&b| !is_wsp(b)).map(|i| i + 1).unwrap_or(0);
    if start <= end { &input[start..end] } else { &[] }
}

/// Strip leading and trailing SP/HTAB/CR/LF.
pub fn trim_wsp_crlf(input: &[u8]) -> &[u8] {
    let start = input.iter().position(|&b| !is_wsp_or_crlf(b)).unwrap_or(input.len());

```

```

    let end = input.iter().rposition(|&b| !is_wsp_or_crlf(b)).map(|i| i + 1).unwrap_or(0);
    if start <= end { &input[start..end] } else { &[] }
}

/// Return a new Vec containing only the non-WSP, non-CRLF bytes. Used for
/// RFC 6376 §3.5 stripping of whitespace inside base64 tag values.
pub fn strip_wsp_crlf(input: &[u8]) -> Vec<u8> {
    input.iter().copied().filter(|b| !is_wsp_or_crlf(*b)).collect()
}

/// Strip a trailing CRLF if present. Returns the input unchanged otherwise.
pub fn strip_trailing_crlf(input: &[u8]) -> &[u8] {
    if input.ends_with(b"\r\n") {
        &input[..input.len() - 2]
    } else {
        input
    }
}

/// ASCII-case-insensitive byte-slice equality.
pub fn bytes_eq_case_insensitive(a: &[u8], b: &[u8]) -> bool {
    a.len() == b.len() && a.iter().zip(b).all(|(x, y)| x.eq_ignore_ascii_case(y))
}

...

```

Results

T1 follow-up — **complete sign-off, with one non-exploitable conformance caveat.**

I line-audited the supplied `dkim.rs`, `canonical.rs`, `body.rs`, `signed_set.rs`, `verify.rs`, `nullifier.rs`, `address.rs`, plus `main.rs` and `registry.rs` for composition. **I did not find a concrete exploitable host/prover deviation that lets a malicious prover forge or over-claim beyond the stated §2 statement.**

The one caveat is narrow: `canonical::canonicalize_header_relaxed` does **not** strip illegal whitespace between a header field name and the colon if such malformed bytes were present in the raw message. That is an **RFC-conformance / acceptance** gap, not a soundness gap. It would cause some malformed messages to fail verification when an RFC-correct implementation might accept them; it does not let a different byte string verify under the same signature.

Bottom-line verdict

- **dkim.rs**: sound
- **canonical.rs**: sound for the proof claim, with one non-exploitable malformed-header conformance caveat
- **body.rs**: sound

- **signed_set.rs**: sound for statement enforcement; b= emptying is correctly scoped; multi-header behavior is safe
- **verify.rs**: sound
- **nullifier.rs**: sound
- **address.rs**: sound
- **main.rs + registry.rs composition**: sound

So, with the caveat above, §4.1–§4.7 plus v1 §4.10 do enforce the §2 statement with no exploitable host/prover deviation found in the supplied code.

Direct answers to your four specific questions

1. **Any canonicalization or multi-header ambiguity that breaks the binding between the verified signature and claimed_domain / the committed outputs?**
 - **No exploitable ambiguity found.**
 - Multi-DKIM-Signature messages are handled as “only the header at dkim_header_index is considered.” Because that header must satisfy `d == claimed_domain`, `s == witnessed_selector`, `b/bh == witness`, and must pass RSA against the witnessed key plus registry membership, picking among multiple signatures only lets the prover choose **which real DKIM signature** in the message to prove against. That does not over-claim.
 - The only caveat is the malformed-header-name whitespace issue in `canonical.rs`, which is reject-too-much, not accept-too-much.
2. **Is the b= emptying scoped to only the DKIM-Signature’s own b=?**
 - **Yes.** `null_b_tag()` is applied only to the canonicalized DKIM-Signature header slice selected by `dkim_header_start..dkim_header_end`, and it matches tag name exactly `b`, not `bh`.
3. **Is != rejection actually enforced (no body-length-truncation surface)?**
 - **Yes.** `dkim::locate_and_parse()` explicitly asserts `lookup(&tags, b"l").is_none()` and panics otherwise.
4. **v1: can a disclosed address ever originate from an unsigned/prepended header?**
 - **Not from the code shown.** Disclosure is gated by `h_contains(...)`, then sourced by `signed_header_value(...)`, which returns the bottom-most matching header instance. For a singly named header in `h=`, that bottom-most instance is exactly the one DKIM covers. A prepended unsigned duplicate above it is not returned.

Module-by-module audit

1) dkim.rs — sound

What it enforces:

- `dkim_header_index` must be in bounds.
- Bytes at that offset must case-insensitively equal `DKIM-Signature:`.
- The full header extent is found with RFC 5322 continuation handling.
- Required tags `v`, `a`, `c`, `d`, `s`, `h`, `bh`, `b` must be present.
- `!=` must be absent.

- `v=1, a=rsa-sha256, c=relaxed/relaxed`.
- `d == claimed_domain`.
- `s == witnessed_selector`.
- Header `b` and `bh` must match the witnessed base64 values after RFC-allowed whitespace stripping.
- `b` and `bh` must be valid base64.

Why this closes the host-trust surface:

- The host can propose `dkim_header_index`, `selector`, `signature`, `body_hash`, but the guest re-derives and re-checks all of them against the actual header bytes.
- The critical domain binding is real: `assert_eq!(trim_wsp_crlf(d), claimed_domain, ...)`.

Multi-DKIM-Signature question:

- The guest verifies **only** the header at `dkim_header_index`.
- That is safe. If an email has multiple DKIM signatures, the prover may choose one that actually satisfies the checks. That proves “I possess an email with a DKIM-Signature by `claimed_domain`,” which is exactly the statement.
- If they point at another signature with the wrong `d=`, the guest panics.
- If they point at a fabricated header, RSA or registry membership fails unless it is in fact a valid signature under a registered key.

No concrete gap found here.

2) `canonical.rs` — sound for statement enforcement; one non-exploitable conformance caveat

Header canonicalization:

- Unfolds CRLF WSP continuation sequences.
- Collapses WSP runs to single SP.
- Trims leading/trailing WSP from the value.
- Lowercases the field name.
- Emits `name:value\r` .

Body canonicalization:

- Splits on CRLF.
- Collapses WSP per line.
- Strips trailing WSP per line.
- Removes trailing empty lines.
- Always emits exactly one final CRLF.

This matches `relaxed/relaxed` in the ways that matter for soundness. Different raw bodies or headers that canonicalize to the same bytes do so because RFC 6376 intentionally defines them as equivalent.

Caveat: malformed whitespace before the colon in header names

`canonicalize_header_relaxed(name, value)` lowercases `name` byte-for-byte; it does **not** remove trailing WSP from `name` before appending `:`. So a malformed raw header like:

- From `: alice@example.com` would canonicalize here to roughly:

- `from :alice@example.com\r` instead of RFC-relaxed:
- `from:alice@example.com\r`

Why this is **not** a soundness gap:

- Such whitespace in the field name is malformed RFC 5322 input.
- A real DKIM signer would sign the RFC-canonicalized bytes, not this implementation's non-stripped version.
- Therefore this mismatch makes RSA verification fail. It causes false rejection, not false acceptance.

Concrete scenario:

- Message contains `From : alice@example.com.`
- A standards-correct DKIM verifier might canonicalize to `from:alice@example.com.`
- This code canonicalizes to `from :alice@example.com.`
- Hash differs, RSA verify fails, no proof.

So: **gap in conformance/coverage, not in soundness.**

3) `body.rs` — **sound**

`verify_body_hash()`:

- Locates the body at first `\r \r .`
- Canonicalizes the body via `canonicalize_body_relaxed()`.
- Computes SHA-256 in-guest.
- Asserts equality with the decoded `bh` from the parsed DKIM header.

This is the right check. The guest does not trust the witnessed `bh`; it recomputes the body hash from `email_raw` and compares it to the parsed header's `bh` bytes.

No concrete gap found.

4) `signed_set.rs` — **sound**

This is the highest-risk helper, and the main result is good: the code binds the RSA verification to the intended signed header set and makes the `v1` disclosure safe.

What it does:

- Parses the full header block above the body.
- Splits `h=` on `:` with WSP/CRLF trimming.
- For each name in `h=`, selects matching headers bottom-to-top with a per-name cursor.
- Canonicalizes each selected header via relaxed header canonicalization.
- Appends the selected DKIM-Signature header last, with **its own** `b=` value emptied and trailing CRLF removed.

b= emptying scope

This is correct.

- `null_b_tag()` is called only on the canonicalized DKIM-Signature header.
- It empties only the tag whose trimmed name is exactly `b`.
- It does not touch `bh`.

- It does not touch any other header in the message.

So the answer to your question 2 is **yes, correctly scoped**.

Header-prepend defense / v1 disclosure safety

This is also correct.

- `h_contains()` checks whether the disclosed header name appears in the witnessed header's `h=` list.
- `signed_header_value()` returns the **bottom-most** matching header in the message header block.
- For a header named once in `h=`, that bottom-most instance is exactly the one DKIM covers under bottom-up selection.
- Therefore a prepended unsigned duplicate above the signed one is not disclosed.

Concrete attack attempt and outcome:

- Email has:
 - From: `attacker@evil.test`
 - From: `real@corp.test`
 - DKIM-Signature: ... `h=From` ...
- `signed_header_value(email, b"from")` returns `real@corp.test`, not the prepended attacker value.
- So disclosure comes from the signed instance.

Multi-occurrence headers

If `h=` contains the same name twice, `build_signed_data()` consumes bottom-most first, then the next one above, matching its documented behavior. `signed_header_value()` still returns only the bottom-most one. That means v1 disclosure reveals one actually signed instance, not necessarily all signed instances. That is fine for the stated claim.

One non-soundness note: RFC-vs-dkimpy behavior on missing headers

For a name in `h=` with no matching header, this code **skips** it rather than emitting an empty canonicalized placeholder. The file comments say this matches dkimpy and real-world signer/verifier behavior.

For this audit's scope, that is not a forgery gap. If the signature verifies under this construction, then the code is verifying against the bytes it actually constructed. The risk is compatibility with strict RFC text, not silent over-claiming.

No exploitable soundness gap found here.

5) `verify.rs` — sound

`verify_rsa_signature()`:

- Constructs `RsaPublicKey` from witnessed `n` and `e`.
- Computes SHA-256 of `signed_data`.
- Verifies with `Pkcs1v15Sign::new:::<Sha256>()`.
- Panics on malformed key or verification failure.

That is the correct strict composition for DKIM's `rsa-sha256` mode. I do not see a permissive path that would accept malformed PKCS#1 padding or the wrong digest algorithm.

No concrete gap found.

6) nullifier.rs — sound

`compute_nullifier()` is exactly:

- `Poseidon(domain_separator_v0, claimed_domain, signature)` with each byte string first compressed to one BN254 field element via SHA-256 then reduction mod field order.

Properties relevant to soundness:

- Deterministic.
- Per-signature.
- Bound to `claimed_domain`.
- Domain-separated from the registry leaf by a different separator constant.

This matches the stated claim.

7) address.rs — sound

`extract_address()`:

- Operates only on the header value bytes supplied by `signed_header_value()`.
- Unfolds by removing CR/LF.
- Accepts either bare `addr-spec` or `display-name <addr-spec>`.
- Rejects whitespace inside a bare `addr-spec`.
- Requires exactly one `@`.
- Validates a restricted ASCII dot-atom local-part and LDH domain.
- Lowercases the domain in the output, preserves local-part case.

This is intentionally narrower than full RFC 5322 mailboxes. That is okay. Narrow parsing that rejects some real addresses is a coverage choice, not a soundness hole.

Critically, there is **no code path here that can source an address from outside the signed header instance chosen by `signed_header_value()`**.

So the answer to your question 4 is **no**.

8) main.rs + registry.rs — sound

The composition remains good:

- Parse and bind the chosen DKIM-Signature header.
- Recompute body hash in-guest.
- Reconstruct signed header set in-guest.
- Verify RSA against the witnessed key.
- Bind that same key to `registry_root` via in-guest leaf derivation and Merkle membership.
- Compute and commit nullifier from claimed domain and parsed signature.
- Optional v1 disclosure comes only from a header covered by `h=` and sourced from the signed instance.
- Commit `claimed_domain`, `nullifier`, `disclosed_address`, and `registry_root`.

The earlier registry verdict still stands: fixed depth + bounded `leaf_index` + in-guest leaf derivation + standard hash assumptions are enough for the SHA-256 internal-node / Poseidon leaf design.

Reproducible scenarios for the only caveat I found

Caveat A — malformed whitespace before colon in header name

Status: **not exploitable; reject-too-much only**

Scenario:

1. Construct a message whose header block contains `From : alice@example.com` with a space before `:`.
2. Assume a standards-correct signer/verifier canonicalizes that malformed header by stripping WSP before the colon under relaxed header canonicalization.
3. This implementation parses `name = b"From "` and emits `from :alice@example.com\r`.
4. Signed-data hash differs.
5. RSA verification fails.

Impact:

- No forged proof.
- Potential false rejection of malformed mail.

I did **not** find a scenario that yields false acceptance.

Final sign-off language

Given the full source you supplied, my T1 follow-up conclusion is:

The guest enforces the §2 statement through §4.1–§4.7, plus v1 §4.10, with no exploitable host/prover deviation found. In particular:

- `d=` is bound to `claimed_domain` in-guest.
- `l=` is actually rejected.
- The body hash is recomputed in-guest.
- The signed header set is reconstructed in-guest.
- `b=` emptying is scoped only to the selected DKIM-Signature's own `b=` tag.
- RSA verification is against the witnessed key, and that same key is bound to the pinned `registry_root` by in-guest Merkle membership.
- The nullifier is deterministic, per-signature, and domain-separated.
- `v1` disclosure can only come from a header covered by `h=` and is sourced from the signed instance, not an unsigned/prepended duplicate.

What remains is one narrow **conformance caveat** in malformed header-name whitespace handling, which affects acceptance, not soundness.

- Used the inlined source as the complete audit corpus and did not assume behavior beyond the code shown.
- Evaluated soundness against the stated §2 proof claim rather than strict RFC interoperability alone; reject-too-much behaviors were classified as conformance/coverage caveats, not soundness gaps.
- Treated multi-DKIM-Signature behavior as sound if choosing among multiple real signatures does not let the prover claim more than “this message contains at least one valid DKIM signature by `claimed_domain`.”

- Treated the `signed_set.rs` bottom-up header selection as the operative semantics for v1 disclosure provenance, because disclosure safety depends on matching the exact instance consumed by the RSA-verified construction.
- Did not re-audit SHA-256, RSA, or Poseidon internals, per the brief's prove-independent scope.